

Matematici Asistate de Calculator

Prof. Dr. Călin-Adrian POPA

Cursul 1

3 Martie 2026

- Facultatea de Automatică și Calculatoare, Universitatea Politehnica Timișoara
 - Inginer, Calculatoare și Tehnologia Informației, 2009
 - Master, Master of Software Engineering, 2011
 - Doctor, Calculatoare și Tehnologia Informației, 2015
 - Asistent, 2015
 - Șef de lucrări (Lector), 2016
 - Conferențiar, 2019
 - Profesor, 2023
- Facultatea de Matematică și Informatică, Universitatea de Vest din Timișoara
 - Licențiat, Matematică, 2013
 - Master, Modelări Analitice și Geometrice ale Sistemelor, 2015

• Curs

- 14 cursuri în primele 10 săptămâni
- 2 examene scrise
 - săptămâna 5
 - săptămâna 11
- N.E. = $\frac{\min(7 \text{ probleme} \times 10 \text{ puncte} + 7 \text{ probleme} \times 10 \text{ puncte} + 14 \text{ puncte pe prezență}, 140)}{14}$
- 2/3 din nota finală

• Laborator

- 10 laboratoare
- 2 teste
 - săptămâna 6
 - săptămâna 12
- N.L. = $\frac{4 \text{ probleme} \times 10 \text{ puncte} + 5 \text{ probleme} \times 10 \text{ puncte} + 10 \text{ puncte din oficiu}}{10}$
- 1/3 din nota finală

• Site

- <https://cv.upt.ro/>

Cuprinsul cursului

- 1 Fundamente
- 2 Rezolvarea ecuațiilor
- 3 Sisteme de ecuații
- 4 Interpolarea
- 5 Cele mai mici pătrate
- 6 Derivarea și integrarea numerică
- 7 Ecuații diferențiale ordinare
- 8 Interpolarea trigonometrică și TFR
- 9 Compresia
- 10 Valori proprii și valori singulare
- 11 Optimizarea

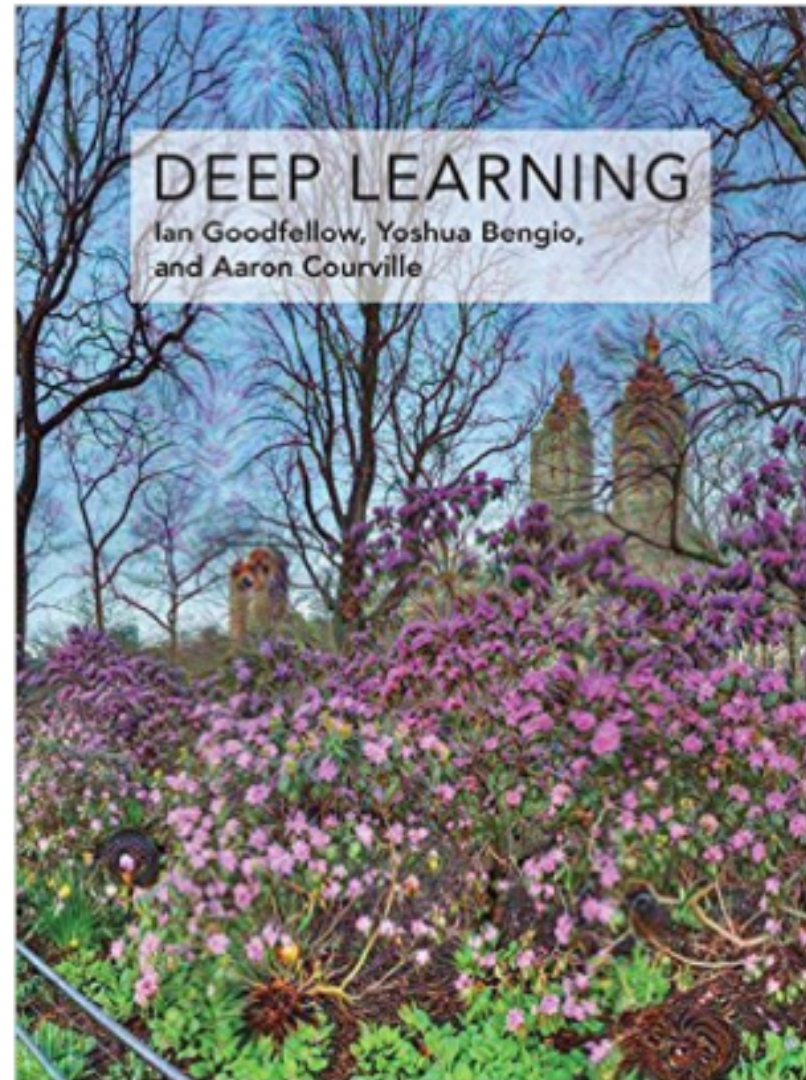
Cuprinsul laboratorului

- 1 Introducere în MATLAB
- 2 Rezolvarea ecuațiilor
- 3 Sisteme de ecuații
- 4 Interpolarea
- 5 Cele mai mici pătrate
- 6 Derivarea și integrarea numerică
- 7 Ecuații diferențiale ordinare
- 8 Interpolarea trigonometrică și TFR. Compresia
- 9 Valori proprii și valori singulare
- 10 Optimizarea

La ce ne folosește?

- fiecare capitol are aplicații în știința calculatoarelor
 - digital signal processing
 - image and video processing
 - computer graphics
 - computer vision
 - machine learning
 - data science
 - deep learning
- MATLAB – limbaj important în mai multe domenii
 - parallel computing
 - statistică, machine learning și optimizare
 - signal processing
 - image processing și computer vision

Deep learning



- Linear Algebra
- Probability and Information Theory
- Numerical Computation

1 Fundamente

- scopul acestui curs este de a prezenta și discuta metode de rezolvare a unor probleme de matematică folosind calculatorul
- operațiile aritmetice fundamentale sunt adunarea și înmulțirea
- acestea sunt și operațiile necesare pentru a evalua un polinom $P(x)$ într-un punct x
- polinoamele stau la baza multor tehnici computaționale pe care le vom construi în acest curs
- este important să știm cum să evaluăm un polinom
- vom studia cum să implementăm evaluarea polinoamelor cât mai eficient posibil

1.1 Evaluarea unui polinom

- care este cea mai bună metodă pentru a evalua polinomul

$$P(x) = 2x^4 + 3x^3 - 3x^2 + 5x - 1,$$

de exemplu, în punctul $x = 1/2$?

- coeficienții polinomului și numărul $1/2$ sunt stocați în memorie, și dorim să minimizăm numărul de adunări și înmulțiri necesare pentru a obține $P(1/2)$
- **Metoda 1.** Prima și cea mai directă metodă este:

$$P\left(\frac{1}{2}\right) = 2 * \frac{1}{2} * \frac{1}{2} * \frac{1}{2} * \frac{1}{2} + 3 * \frac{1}{2} * \frac{1}{2} * \frac{1}{2} - 3 * \frac{1}{2} * \frac{1}{2} + 5 * \frac{1}{2} - 1 = \frac{5}{4}. \quad (1)$$

- numărul de înmulțiri necesare este 10, la care se adaugă 4 adunări
- cu siguranță există o metodă mai bună decât cea din (1)
- numărul de operații poate fi redus prin eliminarea înmulțirii repetate cu $1/2$
- o strategie mai bună este să calculăm mai întâi $(1/2)^4$, memorând și rezultatele parțiale obținute

1.1 Evaluarea unui polinom

- **Metoda 2.** Găsim mai întâi puterile numărului $x = 1/2$, pe care le stocăm pentru eventuale utilizări viitoare:

$$\begin{aligned}\frac{1}{2} * \frac{1}{2} &= \left(\frac{1}{2}\right)^2 \\ \left(\frac{1}{2}\right)^2 * \frac{1}{2} &= \left(\frac{1}{2}\right)^3 \\ \left(\frac{1}{2}\right)^3 * \frac{1}{2} &= \left(\frac{1}{2}\right)^4.\end{aligned}$$

- acum, putem face o simplă însumare a termenilor:

$$P\left(\frac{1}{2}\right) = 2 * \left(\frac{1}{2}\right)^4 + 3 * \left(\frac{1}{2}\right)^3 - 3 * \left(\frac{1}{2}\right)^2 + 5 * \frac{1}{2} - 1 = \frac{5}{4}.$$

- avem 3 înmulțiri ale lui $1/2$, plus alte 4 înmulțiri
- am redus numărul total de înmulțiri la 7, cu aceleași 4 adunări
- este reducerea de la 14 la 11 operații o îmbunătățire semnificativă?
- dacă polinomul trebuie evaluat pentru diferite valori ale lui x , de mai multe ori pe secundă, atunci diferența poate fi esențială

1.1 Evaluarea unui polinom

- **Metoda 3.** (Înmulțirea imbricată) Rescriem polinomul astfel încât să poată fi evaluat din interior spre exterior:

$$\begin{aligned}P(x) &= -1 + x(5 - 3x + 3x^2 + 2x^3) \\&= -1 + x(5 + x(-3 + 3x + 2x^2)) \\&= -1 + x(5 + x(-3 + x(3 + 2x))) \\&= -1 + x * (5 + x * (-3 + x * (3 + x * 2))).\end{aligned}\tag{2}$$

- acum facem evaluarea dinspre interior înspre exterior:

$$\begin{aligned}\text{înmulțim } \frac{1}{2} * 2, & \quad \text{adunăm } + 3 \rightarrow 4 \\ \text{înmulțim } \frac{1}{2} * 4, & \quad \text{adunăm } - 3 \rightarrow -1 \\ \text{înmulțim } \frac{1}{2} * -1, & \quad \text{adunăm } + 5 \rightarrow \frac{9}{2} \\ \text{înmulțim } \frac{1}{2} * \frac{9}{2}, & \quad \text{adunăm } - 1 \rightarrow \frac{5}{4}.\end{aligned}\tag{3}$$

1.1 Evaluarea unui polinom

- această metodă, numită **înmulțire imbricată** sau **metoda lui Horner**, evaluează polinomul folosind 4 înmulțiri și 4 adunări
- în general, un polinom de gradul d poate fi evaluat utilizând această metodă folosind d înmulțiri și d adunări
- acest exemplu este caracteristic întregului domeniu al metodelor computaționale implementate cu ajutorul calculatorului
- în primul rând, calculatoarele fac foarte rapid operații foarte simple
- în al doilea rând, este foarte important ca până și aceste operații simple să fie făcute cât mai eficient, deoarece este posibil ca ele să fie executate de foarte multe ori
- în al treilea rând, cea mai bună metodă s-ar putea să nu fie și cea mai evidentă metodă

1.1 Evaluarea unui polinom

- forma generală a unui polinom $c_1 + c_2x + c_3x^2 + c_4x^3 + c_5x^4$ poate fi scrisă în formă imbricată astfel:

$$c_1 + x(c_2 + x(c_3 + x(c_4 + x(c_5)))), \quad (4)$$

însă anumite aplicații necesită o formă mai generală

- în particular, interpolarea va avea nevoie ca polinomul să fie scris sub forma

$$c_1 + (x - r_1)(c_2 + (x - r_2)(c_3 + (x - r_3)(c_4 + (x - r_4)(c_5)))), \quad (5)$$

unde r_1, r_2, r_3 , și r_4 sunt numite **puncte de bază**

- luând $r_1 = r_2 = r_3 = r_4 = 0$ în (5), obținem forma imbricată din (4)

1.1 Evaluarea unui polinom

Exemplul 1

- găsiți o metodă eficientă pentru evaluarea polinomului $P(x) = 4x^5 + 7x^8 - 3x^{11} + 2x^{14}$
- ideea este de a factoriza x^5 din fiecare termen și de a scrie polinomul în x^3 :

$$\begin{aligned} P(x) &= x^5(4 + 7x^3 - 3x^6 + 2x^9) \\ &= x^5 * (4 + x^3 * (7 + x^3 * (-3 + x^3 * (2))))). \end{aligned}$$

- pentru fiecare punct x , trebuie să calculăm mai întâi $x * x = x^2$, $x * x^2 = x^3$, și $x^2 * x^3 = x^5$
- aceste trei înmulțiri, împreună cu înmulțirea lui x^5 , și cele trei înmulțiri și trei adunări necesare evaluării polinomului de gradul 3 în x^3 dau numărul total de operații necesar evaluării acestui polinom, și anume 7 înmulțiri și 3 adunări

1.2 Numere binare

- pentru a pregăti discuția despre operațiile aritmetice realizate în calculator, trebuie mai întâi să înțelegem sistemul de numerație binar
- numerele zecimale sunt convertite din baza 10 în baza 2 pentru a fi stocate în calculator și a simplifica anumite operații ca adunarea sau înmulțirea
- pentru a furniza rezultatele ca numere zecimale, acest proces este inversat
- în această secțiune vom discuta diverse modalități de conversie între numere zecimale și numere binare
- numerele binare sunt exprimate sub forma

$$\dots b_2 b_1 b_0 . b_{-1} b_{-2} \dots,$$

unde fiecare cifră binară, sau **bit**, este 0 sau 1

- echivalentul în baza 10 a acestui număr binar este

$$\dots b_2 2^2 + b_1 2^1 + b_0 2^0 + b_{-1} 2^{-1} + b_{-2} 2^{-2} \dots$$

- de exemplu, numărul zecimal 4 se exprimă ca $(100)_2$ în baza 2
- $3/4$ este reprezentat ca $(0.11)_2$ în aceeași bază

1.2.1 Conversia din zecimal în binar

- numărul zecimal 53.7 va fi reprezentat ca $(53.7)_{10}$, pentru a sublinia faptul că va fi interpretat ca fiind în baza 10
- pentru a-l converti în binar, cel mai simplu este a fi împărțit în partea întreagă și în partea fracționară, și a converti fiecare parte separat
- pentru numărul $(53.7)_{10} = (53)_{10} + (0.7)_{10}$, vom converti fiecare parte în binar, și vom combina rezultatele
- **Partea întreagă.** Numerele zecimale se convertesc în binar prin împărțirea lor succesivă la 2 și reținerea resturilor acestor împărțiri
- resturile care pot fi 0 sau 1, sunt reținute pornind de la virgula zecimală (sau, mai exact de la **rădăcină**) și înaintând spre stânga
- pentru $(53)_{10}$, vom avea

$$53 \div 2 = 26 \text{ R } 1$$

$$26 \div 2 = 13 \text{ R } 0$$

$$13 \div 2 = 6 \text{ R } 1$$

$$6 \div 2 = 3 \text{ R } 0$$

$$3 \div 2 = 1 \text{ R } 1$$

$$1 \div 2 = 0 \text{ R } 1.$$

1.2.1 Conversia din zecimal în binar

- prin urmare, numărul 53 în baza 10 poate fi scris folosind biți ca 110101, ceea ce se notează $(53)_{10} = (110101)_2$
- verificând rezultatul, avem că $110101 = 2^5 + 2^4 + 2^2 + 2^0 = 32 + 16 + 4 + 1 = 53$
- **Partea fracționară.** Convertim $(0.7)_{10}$ în binar inversând pașii de mai sus
- înmulțim cu 2 succesiv și reținem părțile întregi, pornind de la virgula zecimală și înaintând spre dreapta

$$0.7 \times 2 \quad 0.4 + 1$$

$$0.4 \times 2 \quad 0.8 + 0$$

$$0.8 \times 2 \quad 0.6 + 1$$

$$0.6 \times 2 \quad 0.2 + 1$$

$$0.2 \times 2 \quad 0.4 + 0$$

$$0.4 \times 2 \quad 0.8 + 0$$

⋮

1.2.1 Conversia din zecimal în binar

- observăm că acest proces se repetă după patru pași și se va repeta exact la fel, la infinit
- prin urmare,

$$(0.7)_{10} = (0.1011001100110\dots)_2 = (0.1\overline{0110})_2,$$

unde am folosit notația cu bară deasupra pentru a specifica o secvență de biți care se repetă periodic

- punând împreună cele două părți, concluzionăm că

$$(53.7)_{10} = (110101.1\overline{0110})_2.$$

1.2.2 Conversia din binar în zecimal

- pentru a converti un număr binar în zecimal, este din nou bine să-l despărțim în partea întreagă și partea fracționară
- **Partea întreagă.** Pur și simplu adunăm puterile lui 2 înmulțite cu biții corespunzători respectivelor puteri
- numărul binar $(10101)_2$ este
 $1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (21)_{10}$
- **Partea fracționară.** Dacă partea fracționară este finită, procedăm de aceeași manieră
- de exemplu,

$$(0.1011)_2 = \frac{1}{2} + \frac{1}{8} + \frac{1}{16} = \left(\frac{11}{16}\right)_{10}.$$

- complicația apare atunci când partea fracționară nu este finită, mai exact conține o parte periodică
- cea mai simplă metodă de convertire a unei secvențe binare periodice într-o fracție zecimală este de a folosi proprietatea de deplasare a înmulțirii cu 2
- de exemplu, să presupunem că vrem să convertim numărul binar $x = (0.\overline{1011})_2$ în zecimal

1.2.2 Conversia din binar în zecimal

- înmulțim pe x cu 2^4 , operație care realizează o deplasare cu 4 poziții la stânga în binar
- apoi scădem valoarea inițială a lui x :

$$\begin{aligned}2^4 x &= 1011.\overline{1011} \\ x &= 0000.\overline{1011}.\end{aligned}$$

- obținem astfel

$$(2^4 - 1)x = (1011)_2 = (11)_{10}.$$

- apoi îl scoatem pe x din această ecuație, și obținem
 $x = (0.\overline{1011})_2 = (11/15)_{10}$
- într-un alt exemplu, să presupunem că partea fracționară nu este periodică imediat după virgula zecimală, ca în numărul binar
 $x = (0.10\overline{101})_2$
- înmulțirea cu 2^2 deplasează cu 2 poziții la stânga, rezultând
 $y = 2^2 x = 10.\overline{101}$

1.2.2 Conversia din binar în zecimal

- partea fracționară a lui y , notată $z = 0.\overline{101}$, este calculată ca mai sus:

$$\begin{aligned}2^3 z &= 101.\overline{101} \\ z &= 000.\overline{101}.\end{aligned}$$

- prin urmare, $7z = 5$, și $y = 2 + 5/7$, $x = 2^{-2}y = 19/28$ în baza 10
- numerele binare sunt elementele de bază ale operațiilor aritmetice realizate în calculator, dar ele pot fi lungi și greu de interpretat pentru factorul uman
- este folositor să folosim baza 16 pentru a reprezenta numerele binare mai ușor
- **numerele hexazecimale** sunt reprezentate de cele 16 numerale 0, 1, 2, ..., 9, A, B, C, D, E, F
- fiecare număr hexazecimal poate fi reprezentat prin 4 biți
- astfel, $(1)_{16} = (0001)_2$, $(8)_{16} = (1000)_2$, și $(F)_{16} = (1111)_2 = (15)_{10}$

1.3 Reprezentarea în virgulă flotantă a numerelor reale

- în această secțiune, vom prezenta un model pentru operațiile aritmetice realizate în calculator, folosind numere reprezentate în virgulă flotantă
- vom alege modelul standard de virgulă flotantă IEEE 754
- formatul pentru aritmetica în virgulă flotantă a devenit unul comun în industria calculatoarelor pentru operațiile în simplă precizie și dublă precizie
- erorile de rotunjire sunt comune atunci când locații de memorie cu precizie finită sunt folosite pentru a reprezenta numere reale care au precizie infinită
- deși am spera ca erorile mici care apar într-un calcul lung să aibă doar un efect minor asupra rezultatului, acest lucru nu se întâmplă întotdeauna
- **algoritmi simpli pot amplifica erori microscopice la un nivel macroscopic**

1.3.1 Formate de virgulă flotantă

- standardul IEEE este format dintr-un set de reprezentări binare ale numerelor reale
- un **număr în virgulă flotantă** este format din trei părți: **semnul** (+ sau –), **mantisa**, care conține șirul de biți semnificativi, și **exponentul**
- cele trei părți sunt stocate împreună într-un singur **cuvânt** de calculator
- există trei nivele de precizie utilizate în general pentru numerele în virgulă flotantă: precizia *simplă*, precizia *dublă* și precizia *extinsă*, cunoscută și ca precizia *dublă lungă*
- numărul de biți alocați pentru fiecare număr în virgulă flotantă în cele trei formate este 32, 64, și, respectiv, 80
- biții sunt împărțiți între cele trei părți ale unui număr în virgulă flotantă astfel:

precizia	semnul	exponentul	mantisa
simplă	1	8	23
dublă	1	11	52
dublă lungă	1	15	64

1.3.1 Formate de virgulă flotantă

- toate cele trei tipuri de precizie funcționează practic în același fel
- forma unui număr în virgulă flotantă **normalizat** IEEE este

$$\pm 1.bbb\dots b \times 2^p, \quad (6)$$

unde fiecare dintre cei N de b este 0 sau 1, iar p un număr binar format din M biți, reprezentând exponentul

- normalizarea înseamnă că, după cum se arată în (6), bitul cel mai din stânga (semnificativ) trebuie să fie 1
- când un număr binar este stocat ca un număr în virgulă flotantă normalizat, este aliniat la stânga, ceea ce înseamnă că bitul de 1 aflat cel mai la stânga este deplasat până pe prima poziție dinaintea virgulei
- această deplasare este compensată printr-o schimbare a exponentului
- de exemplu, numărul zecimal 9, care este 1001 în binar, va fi stocat sub forma

$$+1.001 \times 2^3,$$

deoarece o deplasare de 3 biți, care este echivalentă cu o înmulțire cu 2^3 , este necesară pentru a muta bitul de 1 aflat cel mai la stânga până la poziția corectă

1.3.1 Formate de virgulă flotantă

- acest protocol este simplu, însă este părtinitor, în sensul în care întotdeauna deplasează rezultatul către zero
- metoda alternativă este **rotunjirea**
- în baza 10, numerele sunt de obicei rotunjite în sus dacă următoarea cifră este 5 sau mai mare, și rotunjite în jos în caz contrar
- în binar, aceasta corespunde cu rotunjirea în sus dacă bitul este 1 și în jos dacă bitul este 0
- bitul important în dublă precizie este al 53-lea bit de la dreapta virgulei binare, adică primul care se află în afara chenarului, în reprezentarea de mai sus
- tehnica implicită de rotunjire implementată de standardul IEEE este de a aduna un 1 la bitul 52 (rotunjire în sus) dacă bitul 53 este 1, și de a nu face nimic (rotunjire în jos) bitului 52, dacă bitul 53 este 0
- cu o excepție: dacă biții care urmează bitului 52 sunt 10000 . . . , adică exact la jumătate între rotunjirea în sus și rotunjirea în jos, se va face o rotunjire în sus sau în jos, în funcție de care dintre cele două alegeri va face bitul final 52 să fie egal cu 0
- de ce există această situație de excepție care poate părea ciudată?

1.3.1 Formate de virgulă flotantă

- cu excepția acestui caz, regula se traduce prin rotunjirea la cel mai apropiat număr în virgulă flotantă normalizat față de numărul inițial, de unde și numele de regula rotunjirii la cea mai apropiată valoare
- eroarea de rotunjire făcută va fi la fel de probabil să fie în sus sau în jos
- prin urmare, în cazul de excepție, în care există două numere în virgulă flotantă la care se poate face rotunjirea, care se află la distanțe egale față de numărul inițial, decizia va fi luată astfel încât să nu fie preferată rotunjirea în jos sau în sus, în mod sistematic
- această măsură este menită să descurajeze o alunecare ușoară în calcule lungi, datorată unei rotunjiri părtinitoare
- alegerea de a face bitul final 52 egal cu 0 în cazul de egalitate menționat mai sus este într-un fel arbitrară, dar are avantajul de a nu prefera nici rotunjirea în sus, nici rotunjirea în jos

1.3.1 Formate de virgulă flotantă

Algoritmul 1 (Regula IEEE a rotunjirii la cea mai apropiată valoare)

Pentru dubla precizie, dacă al 53-lea bit de la dreapta virgulei binare este 0, atunci rotunjim în jos (trunchiem după bitul 52). Dacă bitul al 53-lea este 1, atunci rotunjim în sus (adunăm 1 la bitul 52), cu excepția cazului în care biții de după 1 sunt 0, caz în care 1 este adunat la bitul 52 dacă și numai dacă bitul 52 este 1.

- pentru numărul 9.4 discutat anterior, cel de-al 53-lea bit din dreapta virgulei binare este 1 și este urmat de alți biți care nu sunt toți zero
- conform regulii rotunjirii la cea mai apropiată valoare, trebuie să facem o rotunjire în sus, adică să adunăm un 1 la bitul 52
- prin urmare, numărul în virgulă flotantă care îl reprezintă pe 9.4 este

$$+1.\boxed{00101100110011001100110011001100110011001100110011001101}_{(7)} \times 2^3.$$

1.3.1 Formate de virgulă flotantă

Definiția 2

Notăm numărul în virgulă flotantă în dublă precizie IEEE asociat lui x , folosind regula rotunjirii la cea mai apropiată valoare, cu **fl(x)**.

- în operațiile aritmetice realizate în calculator, numărul real x este înlocuit cu șirul de biți $\text{fl}(x)$
- conform definiției de mai sus, $\text{fl}(9.4)$ este numărul în reprezentare binară dat de (7)
- am ajuns la această reprezentare în virgulă flotantă înlăturând partea infinită dată de $0.\overline{1100} \times 2^{-52} \times 2^3 = 0.\overline{0110} \times 2^{-51} \times 2^3 = 0.4 \times 2^{-48}$ din capătul din dreapta al numărului, și apoi adunând $2^{-52} \times 2^3 = 2^{-49}$ în pasul de rotunjire
- prin urmare,

$$\begin{aligned}\text{fl}(9.4) &= 9.4 + 2^{-49} - 0.4 \times 2^{-48} \\ &= 9.4 + (1 - 0.8)2^{-49} \\ &= 9.4 + 0.2 \times 2^{-49}.\end{aligned}\tag{8}$$

1.3.1 Formate de virgulă flotantă

- cu alte cuvinte, un calculator care folosește reprezentarea în dublă precizie și regula rotunjirii la cea mai apropiată valoare face o eroare de 0.2×2^{-49} atunci când stochează numărul 9.4
- vom spune că 0.2×2^{-49} este **eroarea de rotunjire**
- ceea ce trebuie reținut este faptul că numărul în virgulă flotantă care îl reprezintă pe 9.4 nu este egal cu 9.4, deși este foarte aproape de această valoare
- pentru a cuantifica această apropiere, vom folosi definiția standard a erorii

Definiția 3

Fie x_c o versiune calculată a valorii exacte x . Atunci

$$\text{eroarea absolută} = |x_c - x|,$$

și

$$\text{eroarea relativă} = \frac{|x_c - x|}{|x|},$$

dacă această din urmă cantitate există ($x \neq 0$).

1.3.1 Formate de virgulă flotantă

Algoritmul 2 (Eroarea relativă de rotunjire)

În cadrul standardului IEEE, eroarea relativă de rotunjire $fl(x)$ este mai mică decât jumătate din numărul epsilon mașină:

$$\frac{|fl(x) - x|}{|x|} \leq \frac{1}{2} \epsilon_{mach}. \quad (9)$$

- în cazul numărului $x = 9.4$, am găsit eroarea de rotunjire în (8), care trebuie să satisfacă (9):

$$\frac{|fl(9.4) - 9.4|}{9.4} = \frac{0.2 \times 2^{-49}}{9.4} = \frac{8}{47} \times 2^{-52} < \frac{1}{2} \epsilon_{mach}.$$

1.3.1 Formate de virgulă flotantă

- prin urmare,

$$\begin{aligned}\text{fl}(0.4) &= 0.4 - 2^{-55} - 0.4 \times 2^{-56} + 2^{-54} \\ &= 0.4 + 2^{-54}(-1/2 - 0.1 + 1) \\ &= 0.4 + 2^{-54}(0.4) \\ &= 0.4 + 0.1 \times 2^{-52}.\end{aligned}$$

- observăm că eroarea relativă de rotunjire pentru 0.4 este $0.1/0.4 \times \epsilon_{\text{mach}} = 1/4 \times \epsilon_{\text{mach}}$, conform cu (9)

1.3.2 Reprezentarea numerelor în calculator

- până acum am descris reprezentarea în virgulă flotantă în abstract
- vom prezenta în continuare mai multe detalii despre cum este această reprezentare implementată într-un calculator
- vom discuta formatul în dublă precizie în această subsecțiune, celelalte formate fiind asemănătoare
- fiecărui număr în virgulă flotantă în dublă precizie îi este asignat un cuvânt de 8 octeți (1 octet = 8 biți), sau 64 de biți, pentru a stoca cele trei părți ale sale
- fiecare astfel de cuvânt are forma

$$\boxed{se_1 e_2 \dots e_{11} b_1 b_2 \dots b_{52}}, \quad (10)$$

unde mai întâi este stocat semnul, urmat de 11 biți reprezentând exponentul și de 52 de biți care urmează virgulei zecimale, care reprezintă mantisa

- bitul de semn s este 0 pentru un număr pozitiv și 1 pentru un număr negativ
- cei 11 biți reprezentând exponentul vin din numărul binar pozitiv care rezultă din adunarea lui $2^{10} - 1 = 1023$ la exponent, cel puțin pentru exponenți între -1022 și 1023

1.3.2 Reprezentarea numerelor în calculator

- aceasta acoperă valorile lui $e_1 \dots e_{11}$ de la 1 la 2046, lăsând valorile 0 și 2047 pentru scopuri speciale, care vor fi detaliate ulterior
- numărul 1023 se numește **biasul exponent** al formatului în dublă precizie
- este folosit pentru a converti atât exponenții pozitivi cât și pe cei negativi în numere binare pozitive pentru a fi stocate în biții de exponent
- pentru precizia simplă și dublă lungă, valorile biasului exponent sunt 127 și respectiv 16383
- formatul hexazecimal constă din exprimarea celor 64 de biți ai reprezentării în virgulă flotantă (10) ca 16 numere hexazecimale
- prin urmare, primele 3 numerale hexazecimale reprezintă semnul și exponentul combinate, în timp ce ultimele 13 conțin mantisa

1.3.2 Reprezentarea numerelor în calculator

Exemplul 3

- găsiți reprezentarea în virgulă flotantă în dublă precizie a numărului real 9.4
- din (7), avem că semnul este $s = 0$, exponentul este 3, și cei 52 de biți ai mantisei de după virgula zecimală sunt

0010	1100	1100	1100	1100	1100	1100	1100	1100	1100	1100	1100	1101
------	------	------	------	------	------	------	------	------	------	------	------	------

 $\rightarrow (2CCCCCCCCCCCCD)_{16}$.

- adunând 1023 la exponent obținem $1026 = 2^{10} + 2$, sau $(10000000010)_2$
- combinația semn – exponent este $(010000000010)_2 = (402)_{16}$, dând formatul hexazecimal $4022CCCCCCCCCCCCD$

1.3.2 Reprezentarea numerelor în calculator

- ne întoarcem acum la valorile speciale ale exponentului 0 și 2047
- ultima, 2047, este folosită pentru a reprezenta ∞ dacă șirul de biți ai mantisei este format doar din zerouri și NaN, adică Not a Number, altfel
- deoarece 2047 este reprezentat prin unsprezece biți de 1, sau $e_1 e_2 \dots e_{11} = (111\ 1111\ 1111)_2$, primii doisprezece biți ai lui Inf și -Inf sunt 011111111111 și, respectiv, 111111111111, și cei 52 de biți rămași (mantisa) sunt zero
- numărul NaN începe de asemenea cu 111111111111 dar are o mantisă nenulă
- putem rezuma cele de mai sus în tabelul următor:

numărul	exemplu	format hexazecimal
+Inf	1/0	7FF0000000000000
-Inf	-1/0	FFF0000000000000
NaN	0/0	FFFxxxxxxxxxxxxxx

unde x-urile denotă biți care nu sunt toți egali cu zero

1.3.2 Reprezentarea numerelor în calculator

- trebuie făcută distincția între cel mai mic număr reprezentabil 2^{-1074} și $\epsilon_{\text{mach}} = 2^{-52}$
- sunt multe numere mai mici decât ϵ_{mach} care sunt reprezentabile, chiar dacă adunându-le la 1 nu vom obține niciun efect
- pe de altă parte, numerele în dublă precizie mai mici decât 2^{-1074} nu pot fi reprezentate deloc
- numerele subnormale includ și cel mai important număr, 0
- de fapt, reprezentarea subnormală include două numere în virgulă flotantă diferite, $+0$ și -0 , care sunt tratate în calcule ca fiind același număr real
- reprezentarea în virgulă flotantă a lui $+0$ are bitul de semn $s = 0$, biții de exponent $e_1 \dots e_{11} = 00000000000$, și mantisa 52 de zerouri; pe scurt, toți cei 64 de biți sunt zero
- formatul hexazecimal pentru $+0$ este 00000000000000000000
- pentru numărul -0 , totul este exact la fel, cu excepția bitului de semn $s = 1$
- formatul hexazecimal pentru -0 este 80000000000000000000

1.3.3 Adunarea numerelor în virgulă flotantă

- observăm că 2^{-53} este cel mai mare număr în virgulă flotantă cu această proprietate; orice alt număr adunat cu 1 va rezulta într-o sumă mai mare decât 1 în urma adunării din calculator
- faptul că $\epsilon_{\text{mach}} = 2^{-52}$ nu înseamnă că numerele mai mici decât ϵ_{mach} sunt neglijabile în modelul IEEE
- atâta timp cât acestea sunt reprezentabile în model, calculele cu numere de această mărime sunt la fel de exacte, presupunând că acestea nu sunt adunate la sau scăzute din 1
- este important să se înțeleagă că aritmetica din calculator, din cauza trunchierii și a rotunjirii pe care le efectuează, poate da uneori rezultate surprinzătoare

1.3.3 Adunarea numerelor în virgulă flotantă

- de exemplu, dacă unui calculator care folosește regula IEEE a rotunjirii la cea mai apropiată valoare îi este dată instrucțiunea de a stoca numărul 9.4, apoi de a scădea 9 din acesta, și apoi de a scădea 0.4 din rezultat, ceea ce se obține va fi diferit de zero!
- ceea ce se întâmplă este că: mai întâi 9.4 este stocat ca $9.4 + 0.2 \times 2^{-49}$, după cum s-a arătat anterior
- când 9 este scăzut din această valoare (observăm că 9 poate fi reprezentat fără eroare), rezultatul este $0.4 + 0.2 \times 2^{-49}$
- acum, dând calculatorului instrucțiunea de a scădea 0.4 din acest număr, rezultă în scăderea numărului $\text{fl}(0.4) = 0.4 + 0.1 \times 2^{-52}$ (după cum am arătat în Exemplul 2), ceea ce va da

$$0.2 \times 2^{-49} - 0.1 \times 2^{-52} = 0.1 \times 2^{-52}(2^4 - 1) = 3 \times 2^{-53},$$

în loc de zero

- acesta este un număr mic, de ordinul lui ϵ_{mach} , dar nu este zero

1.3.3 Adunarea numerelor în virgulă flotantă

Exemplul 4

- găsiți suma în virgulă flotantă în dublă precizie $(1 + 3 \times 2^{-53}) - 1$
- bineînțeles, în aritmetica reală, răspunsul este 3×2^{-53}
- însă calculele în aritmetica în virgulă flotantă pot da un rezultat diferit
- observăm că $3 \times 2^{-53} = 2^{-52} + 2^{-53}$
- prima adunare este

[illegible]

2 Rezolvarea ecuațiilor

- rezolvarea ecuațiilor este una dintre problemele cele mai de bază în analiza numerică
- acest capitol introduce mai multe metode iterative pentru localizarea soluțiilor x ale ecuației $f(x) = 0$, ele având o mare importanță practică
- se poate pune întrebarea: de ce este necesar să cunoaștem mai mult de o metodă pentru rezolvarea ecuațiilor?
- adesea, alegerea metodei va depinde de costul computațional al evaluării funcției f și probabil și al derivatei ei
- dacă $f(x) = e^x - \sin x$, va dura mai puțin decât o milionime de secundă pentru a determina $f(x)$, și derivata ei este disponibilă, dacă avem nevoie de ea
- dacă $f(x)$ reprezintă temperatura de îngheț a unei soluții de etilen glicol la o presiune de x atmosfere, fiecare evaluare a funcției va necesita un timp considerabil într-un laborator bine echipat, iar determinarea derivatei s-ar putea să nu fie fezabilă

2.1 Metoda biseecției

- să ne gândim cum căutăm un nume într-o carte de telefon
- pentru a căuta numele “Ion”, începem prin a deschide cartea la cea mai bună aproximație, să zicem, litera G
- apoi s-ar putea să întoarcem un set de pagini și să ajungem la litera K
- acum putem spune că am găsit limitele între care se află numele Ion, și va trebui să-l căutăm între limite din ce în ce mai mici, care până la urmă vor converge la acest nume
- metoda biseecției reprezintă acest tip de raționament, realizat cât mai eficient posibil

2.1.1 Găsirea limitelor între care se află o rădăcină

Definiția 4

Funcția $f(x)$ are o **rădăcină** în $x = r$ dacă $f(r) = 0$.

- primul pas în rezolvarea unei ecuații este de a verifica dacă există o rădăcină
- un mod de a ne asigura de aceasta este de a găsi limitele între care se află o rădăcină, și anume de a găsi un interval $[a, b]$ pe dreapta reală pentru care un număr din perechea $\{f(a), f(b)\}$ este pozitiv și celălalt este negativ
- acest fapt poate fi exprimat ca $f(a)f(b) < 0$
- dacă f este o funcție continuă, atunci va exista o rădăcină, și anume un r între a și b pentru care $f(r) = 0$
- acest fapt este rezumat în următorul corolar al teoremei valorii intermediare:

2.1.1 Găsirea limitelor între care se află o rădăcină

Teorema 1 (Teorema valorii intermediare)

Fie f o funcție continuă pe intervalul $[a, b]$. Atunci f ia orice valoare între $f(a)$ și $f(b)$. Mai precis, dacă y este un număr între $f(a)$ și $f(b)$, atunci există un număr c care satisface $a \leq c \leq b$ astfel încât $f(c) = y$.

Teorema 2

Fie f o funcție continuă pe $[a, b]$, care satisface $f(a)f(b) < 0$. Atunci f are o rădăcină între a și b , adică există un număr r care satisface $a < r < b$ și $f(r) = 0$.

2.1.1 Găsirea limitelor între care se află o rădăcină

- în Figura 1, $f(0)f(1) = (-1)(1) < 0$
- există o rădăcină imediat la stânga lui 0.7
- cum putem rafina presupunerea inițială despre locația rădăcinii cu mai multe zecimale exacte?

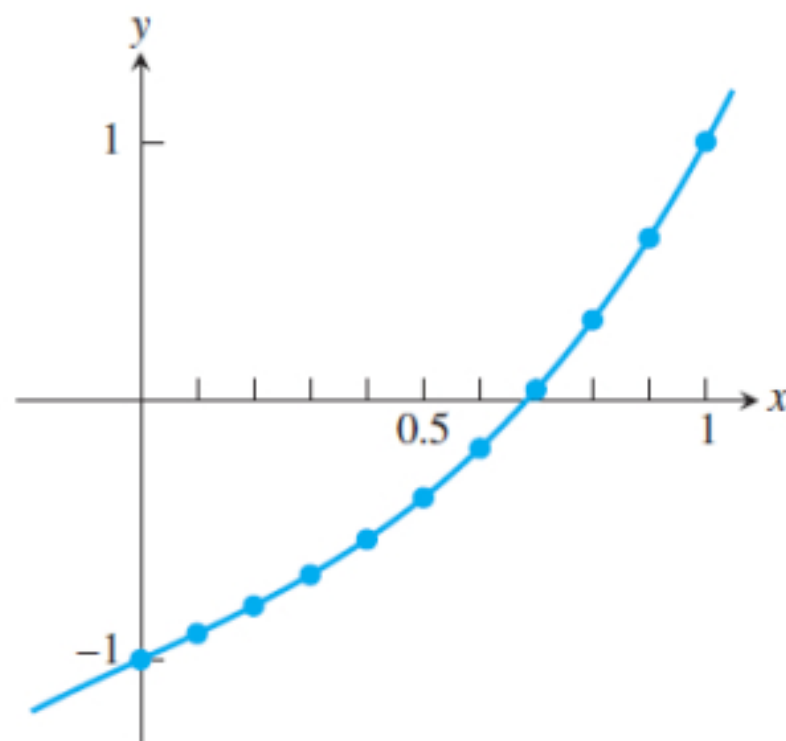


Figura 1: Graficul funcției $f(x) = x^3 + x - 1$. Funcția are o rădăcină între 0.6 și 0.7.

2.1.1 Găsirea limitelor între care se află o rădăcină

- ne vom inspira din felul în care ochiul nostru găsește o soluție atunci când i se dă graficul unei funcții
- este puțin probabil că vom începe căutarea în capătul din stânga al intervalului și vom înainta spre dreapta, oprindu-ne când găsim rădăcina
- probabil un model mai bun pentru ceea ce se întâmplă este că ochiul decide mai întâi asupra unei locații generale, cum ar fi de exemplu că rădăcina este în partea stângă sau în partea dreaptă a intervalului
- atunci continuă prin a decide mai exact cât de departe la dreapta sau la stânga se află rădăcina și îmbunătățește treptat precizia, la fel ca în căutarea unui nume în agenda telefonică
- această abordare generală devine destul de specifică în cadrul metodei bisecției, prezentată în Figura 2

2.1.1 Găsirea limitelor între care se află o rădăcină

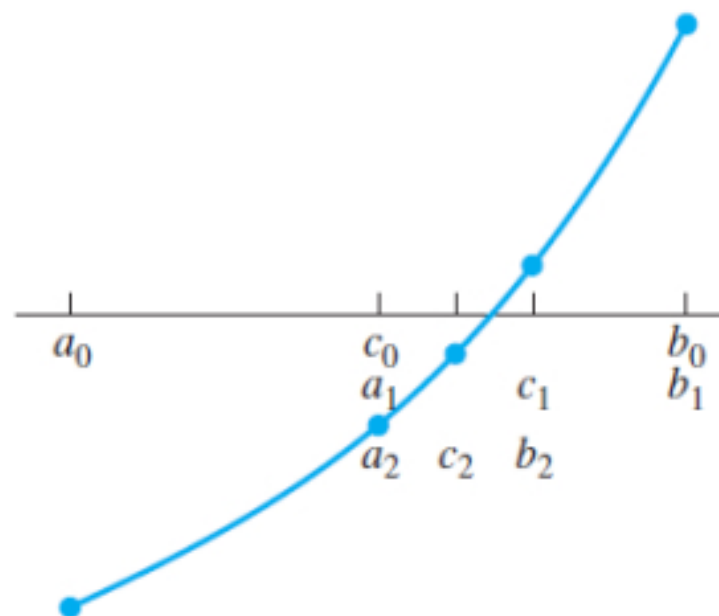


Figura 2: Metoda biseecției. În primul pas, semnul lui $f(c_0)$ este verificat. Deoarece $f(c_0)f(b_0) < 0$, asignăm $a_1 = c_0$, $b_1 = b_0$, și intervalul este înlocuit cu jumătatea lui dreaptă $[a_1, b_1]$. În al doilea pas, subintervalul este înlocuit cu jumătatea lui stângă $[a_2, b_2]$.

2.1.1 Găsirea limitelor între care se află o rădăcină

Algoritmul 3 (Metoda biseției)

Dându-se un interval inițial $[a, b]$ astfel încât $f(a)f(b) < 0$

while $(b - a)/2 > \text{TOL}$

$c = (a + b)/2$

if $f(c) = 0$, **stop**, **end**

if $f(a)f(c) < 0$

$b = c$

else

$a = c$

end

end

Intervalul final $[a, b]$ conține o rădăcină.

Aproximarea rădăcinii este $(a + b)/2$.

2.1.1 Găsirea limitelor între care se află o rădăcină

- verificăm valoarea funcției la mijlocul $c = (a + b)/2$ al intervalului
- deoarece $f(a)$ și $f(b)$ au semne opuse, ori $f(c) = 0$ (în care caz am găsit o rădăcină și suntem gata), ori semnul lui $f(c)$ este opus semnului lui $f(a)$ sau $f(b)$
- dacă $f(c)f(a) < 0$, de exemplu, suntem asigurați că o soluție se află în intervalul $[a, c]$, a cărui lungime este jumătate din cea a intervalului inițial $[a, b]$
- dacă, însă, $f(c)f(b) < 0$, atunci putem spune același lucru despre intervalul $[c, b]$
- în ambele cazuri, un pas reduce problema la găsirea unei rădăcini pe un interval care are lungimea egală cu jumătate din lungimea intervalului inițial
- acest pas poate fi repetat pentru a localiza rădăcina din ce în ce mai exact
- la fiecare pas, limitele între care se află rădăcina sunt altele, reducând incertitudinea despre localizarea soluției, pe măsură ce intervalul devine din ce în ce mai mic

2.1.1 Găsirea limitelor între care se află o rădăcină

Exemplul 5

- găsiți o rădăcină a funcției $f(x) = x^3 + x - 1$ utilizând metoda biseecției pe intervalul $[0, 1]$
- observăm că $f(a_0)f(b_0) = (-1)(1) < 0$, prin urmare o rădăcină există în acest interval
- mijlocul intervalului este $c_0 = 1/2$
- primul pas constă în evaluarea lui $f(1/2) = -3/8 < 0$ și alegerea noului interval ca fiind $[a_1, b_1] = [1/2, 1]$, deoarece $f(1/2)f(1) < 0$
- al doilea pas constă în evaluarea lui $f(c_1) = f(3/4) = 11/64 > 0$, conducând la noul interval $[a_2, b_2] = [1/2, 3/4]$

2.1.1 Găsirea limitelor între care se află o rădăcină

- continuând în această manieră, obținem următoarele intervale:

i	a_i	$f(a_i)$	c_i	$f(c_i)$	b_i	$f(b_i)$
0	0.0000	—	0.5000	—	1.0000	+
1	0.5000	—	0.7500	+	1.0000	+
2	0.5000	—	0.6250	—	0.7500	+
3	0.6250	—	0.6875	+	0.7500	+
4	0.6250	—	0.6562	—	0.6875	+
5	0.6562	—	0.6719	—	0.6875	+
6	0.6719	—	0.6797	—	0.6875	+
7	0.6797	—	0.6836	+	0.6875	+
8	0.6797	—	0.6816	—	0.6836	+
9	0.6816	—	0.6826	+	0.6836	+

2.1.1 Găsirea limitelor între care se află o rădăcină

- concluzionăm din tabel că soluția se află între $a_9 \approx 0.6816$ și $c_9 \approx 0.6826$
- mijlocul acestui interval este $c_{10} \approx 0.6821$, care reprezintă aproximarea rădăcinii
- deși problema a fost să găsim o rădăcină, am găsit de fapt un interval $[0.6816, 0.6826]$ care conține o rădăcină; cu alte cuvinte, rădăcina este $r = 0.6821 \pm 0.0005$
- va trebui să fim mulțumiți cu o aproximare
- bineînțeles, această aproximare poate fi îmbunătățită, dacă este necesar, prin efectuarea mai multor pași din metoda biseecției

2.1.1 Găsirea limitelor între care se află o rădăcină

- la fiecare pas al metodei bisecției, calculăm mijlocul $c_i = (a_i + b_i)/2$ al intervalului curent $[a_i, b_i]$, calculăm $f(c_i)$, și comparăm semnele
- dacă $f(c_i)f(a_i) < 0$, asignăm $a_{i+1} = a_i$ și $b_{i+1} = c_i$
- dacă, din contră, $f(c_i)f(a_i) > 0$, asignăm $a_{i+1} = c_i$ și $b_{i+1} = b_i$
- fiecare pas necesită o nouă evaluare a funcției f și împarte intervalul care conține o rădăcină, reducându-i lungimea cu un factor de 2
- după n pași de calcul al lui c și $f(c)$, am realizat $n + 2$ evaluări de funcție, și cea mai bună estimare a soluției este mijlocul ultimului interval obținut

2.1.2 Cât de exactă și cât de rapidă este metoda bisecției?

- dacă $[a, b]$ este intervalul inițial, atunci după n pași ai metodei bisecției, intervalul $[a_n, b_n]$ are lungimea $(b - a)/2^n$
- alegând mijlocul $x_c = (a_n + b_n)/2$ obținem o aproximare a soluției r , care este la jumătate din lungimea intervalului de soluția adevărată
- rezumând, după n pași din metoda bisecției, avem că

$$\text{Eroarea soluției} = |x_c - r| < \frac{b - a}{2^{n+1}}, \quad (12)$$

$$\text{Evaluări de funcție} = n + 2. \quad (13)$$

- o modalitate bună de a evalua eficiența metodei bisecției este de a ne întreba câtă acuratețe poate fi adusă de fiecare evaluare de funcție
- fiecare pas, sau fiecare evaluare de funcție, reduce incertitudinea în găsirea rădăcinii cu un factor de 2

Definiția 3

O soluție este **corectă cu p zecimale exacte** dacă eroarea este mai mică decât 0.5×10^{-p} .

2.1.2 Cât de exactă și cât de rapidă este metoda bisecției?

Exemplul 6

- folosiți metoda bisecției pentru a găsi o rădăcină a funcției $f(x) = \cos x - x$ pe intervalul $[0, 1]$ cu 6 zecimale exacte
- în primul rând, vom decide câți pași ai metodei bisecției sunt necesari
- conform cu (12), eroarea după n pași este $(b - a)/2^{n+1} = 1/2^{n+1}$
- din definiția celor p zecimale exacte, trebuie ca

$$\frac{1}{2^{n+1}} < 0.5 \times 10^{-6}$$

$$n > \frac{6}{\log_{10} 2} \approx \frac{6}{0.301} = 19.9.$$

- prin urmare, $n = 20$ pași vor fi necesari

2.1.2 Cât de exactă și cât de rapidă este metoda bisecției?

- aplicând metoda bisecției, obținem următorul tabel:

k	a_k	$f(a_k)$	c_k	$f(c_k)$	b_k	$f(b_k)$
0	0.000000	+	0.500000	+	1.000000	—
1	0.500000	+	0.750000	—	1.000000	—
2	0.500000	+	0.625000	+	0.750000	—
3	0.625000	+	0.687500	+	0.750000	—
4	0.687500	+	0.718750	+	0.750000	—
5	0.718750	+	0.734375	+	0.750000	—
6	0.734375	+	0.742188	—	0.750000	—
7	0.734375	+	0.738281	+	0.742188	—
8	0.738281	+	0.740234	—	0.742188	—
9	0.738281	+	0.739258	—	0.740234	—
10	0.738281	+	0.738770	+	0.739258	—
11	0.738769	+	0.739014	+	0.739258	—
12	0.739013	+	0.739136	—	0.739258	—
13	0.739013	+	0.739075	+	0.739136	—

2.1.2 Cât de exactă și cât de rapidă este metoda bisecției?

k	a_k	$f(a_k)$	c_k	$f(c_k)$	b_k	$f(b_k)$
14	0.739074	+	0.739105	—	0.739136	—
15	0.739074	+	0.739090	—	0.739105	—
16	0.739074	+	0.739082	+	0.739090	—
17	0.739082	+	0.739086	—	0.739090	—
18	0.739082	+	0.739084	+	0.739086	—
19	0.739084	+	0.739085	—	0.739086	—
20	0.739084	+	0.739085	—	0.739085	—

- aproximarea rădăcinii cu șase zecimale exacte este 0.739085

2.1.2 Cât de exactă și cât de rapidă este metoda bisecției?

- pentru metoda bisecției, întrebarea câți pași să facem este una simplă—trebuie doar să alegem precizia dorită și să găsim numărul de pași necesari, ca în (12)
- vom vedea că unii algoritmi mai puternici sunt adesea mai puțin predictibili și nu au un analog al (12)
- în acele cazuri, va fi nevoie să stabilim anumite „criterii de oprire” care vor determina circumstanțele în care algoritmul se va opri
- chiar și pentru metoda bisecției, precizia finită a aritmeticii în calculator va pune o limită numărului posibil de zecimale exacte

Vă mulțumesc!